

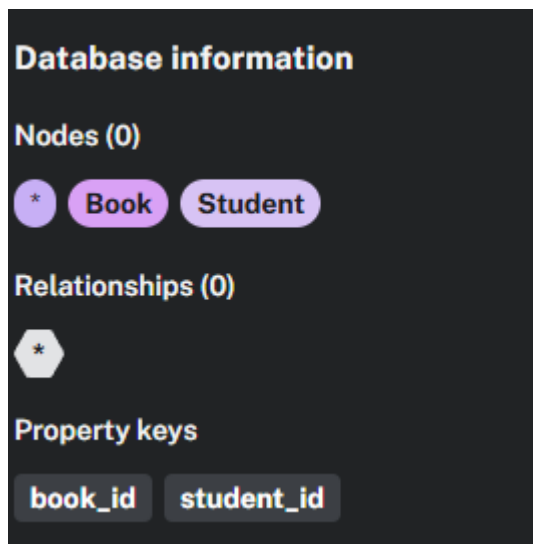
นายปภาณต์ วงษ์ท่าเรือ 664245056 66/44

หัวข้อ: Neo4j และ Graph Database

สร้าง Constraint ของตัวระบุ

```
CREATE CONSTRAINT student_id_unique IF NOT EXISTS  
FOR (s:Student) REQUIRE s.student_id IS UNIQUE;
```

```
CREATE CONSTRAINT book_id_unique IF NOT EXISTS  
FOR (b:Book) REQUIRE b.book_id IS UNIQUE;
```



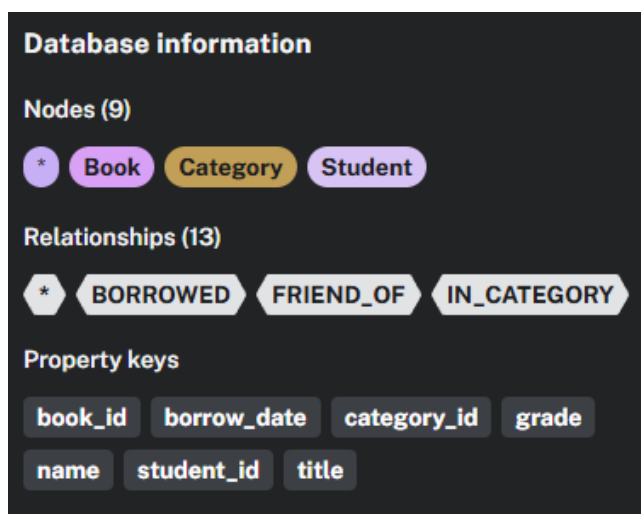
สร้างข้อมูลตัวอย่าง

```
CREATE  
  
(s1:Student {student_id:'S001', name:'Anan', grade:'M.6'}),  
(s2:Student {student_id:'S002', name:'Mali', grade:'M.6'}),  
(s3:Student {student_id:'S003', name:'Somchai', grade:'M.5'}),  
(s4:Student {student_id:'S004', name:'Nida', grade:'M.6'}),  
(b1:Book {book_id:'B101', title:'Python for Beginners'}),  
(b2:Book {book_id:'B102', title:'Artificial Intelligence Basics'}),
```

```

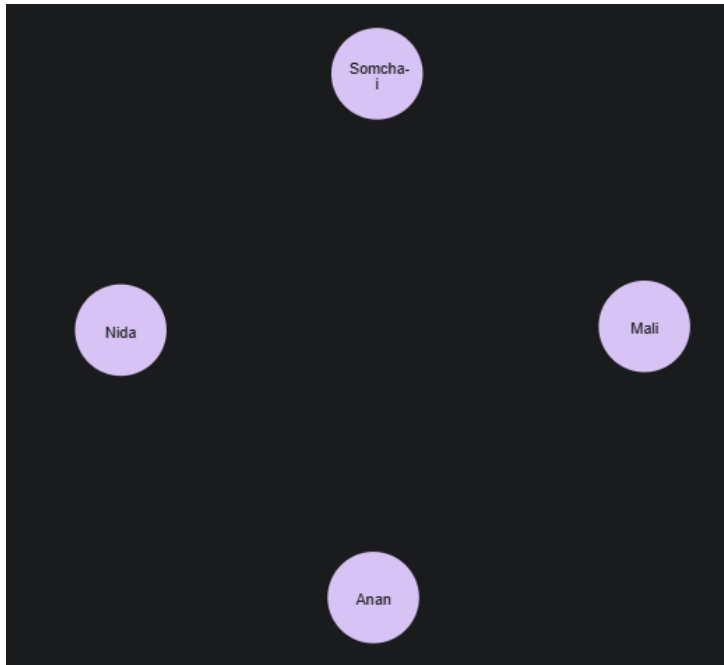
(b3:Book {book_id:'B103', title:'Data Science for Students'}),
(b4:Book {book_id:'B104', title:'Graph Thinking'}),
(c1:Category {category_id:'C01', name:'Computer'}),
(s1)-[:FRIEND_OF]->(s2),
(s1)-[:FRIEND_OF]->(s3),
(s1)-[:FRIEND_OF]->(s4),
(s1)-[:BORROWED {borrow_date:date('2026-09-01')}]>(b1),
(s2)-[:BORROWED {borrow_date:date('2026-09-02')}]>(b2),
(s2)-[:BORROWED {borrow_date:date('2026-09-03')}]>(b4),
(s3)-[:BORROWED {borrow_date:date('2026-09-04')}]>(b3),
(s3)-[:BORROWED {borrow_date:date('2026-09-05')}]>(b4),
(s4)-[:BORROWED {borrow_date:date('2026-09-06')}]>(b2),
(b1)-[:IN_CATEGORY]->(c1),
(b2)-[:IN_CATEGORY]->(c1),
(b3)-[:IN_CATEGORY]->(c1),
(b4)-[:IN_CATEGORY]->(c1);

```



ดูนักเรียนทั้งหมด

```
MATCH (s:Student) RETURN s;
```



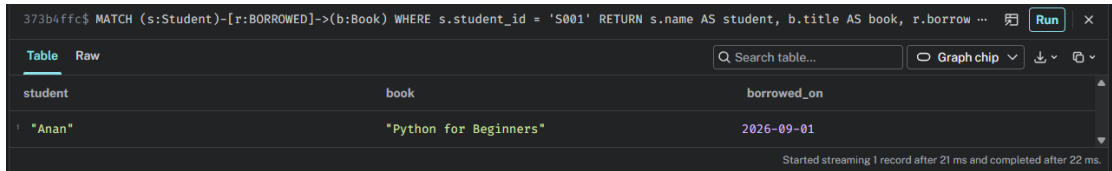
ดูว่าใครยืมหนังสืออะไร

```
MATCH (s:Student)-[r:BORROWED]->(b:Book) RETURN s.name, b.title, r.borrow_date;
```

373b4ffc\$ MATCH (s:Student)-[r:BORROWED]->(b:Book) RETURN s.name, b.title, r.borrow_date;			...	🔍	Run	×
Table	Raw	Q Search table...				
	s.name	b.title	r.borrow_date			
1	"Anan"	"Python for Beginners"	2026-09-01			
2	"Mali"	"Artificial Intelligence Basics"	2026-09-02			
3	"Mali"	"Graph Thinking"	2026-09-03			
4	"Somchai"	"Data Science for Students"	2026-09-04			
5	"Somchai"	"Graph Thinking"	2026-09-05			
6	"Nida"	"Artificial Intelligence Basics"	2026-09-06			

ค้นหาหนังสือที่SO01 เคยยืม

```
MATCH (s:Student)-[r:BORROWED]->(b:Book) WHERE s.student_id = 'SO01' RETURN s.name AS student, b.title AS book, r.borrow_date AS borrowed_on;
```



The screenshot shows a query execution interface with a table of results. The query is: `373b4ffc$ MATCH (s:Student)-[r:BORROWED]->(b:Book) WHERE s.student_id = 'SO01' RETURN s.name AS student, b.title AS book, r.borrow_date AS borrowed_on;`

student	book	borrowed_on
"Anan"	"Python for Beginners"	2026-09-01

Started streaming 1 record after 21 ms and completed after 22 ms.

1 Hop: - เพื่อนของ SO01 คือใคร?

```
MATCH (me:Student {student_id:'SO01'})-[:FRIEND_OF]-(friend:Student) RETURN friend.student_id, friend.name;
```



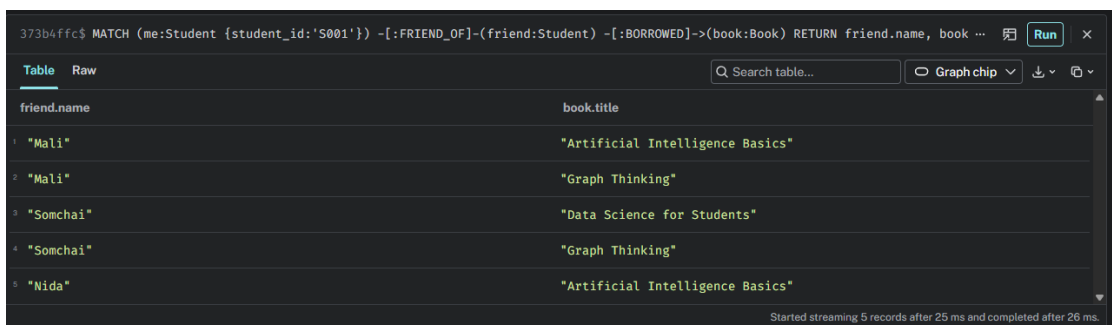
The screenshot shows a query execution interface with a table of results. The query is: `373b4ffc$ MATCH (me:Student {student_id:'SO01'})-[:FRIEND_OF]-(friend:Student) RETURN friend.student_id, friend.name;`

friend.student_id	friend.name
"S002"	"Mali"
"S003"	"Somchai"
"S004"	"Nida"

Started streaming 3 records after 27 ms and completed after 28 ms.

2 Hop: - เพื่อนของ SO01 เคยยืมหนังสืออะไร?

```
MATCH (me:Student {student_id:'SO01'}) -[:FRIEND_OF]-(friend:Student) -[:BORROWED]->(book:Book) RETURN friend.name, book.title;
```



The screenshot shows a query execution interface with a table of results. The query is: `373b4ffc$ MATCH (me:Student {student_id:'SO01'}) -[:FRIEND_OF]-(friend:Student) -[:BORROWED]->(book:Book) RETURN friend.name, book.title;`

friend.name	book.title
"Mali"	"Artificial Intelligence Basics"
"Mali"	"Graph Thinking"
"Somchai"	"Data Science for Students"
"Somchai"	"Graph Thinking"
"Nida"	"Artificial Intelligence Basics"

Started streaming 5 records after 25 ms and completed after 26 ms.

นับว่าหนังสือแต่ละเล่มถูกยืมโดยเพื่อนของ S001 ที่คน

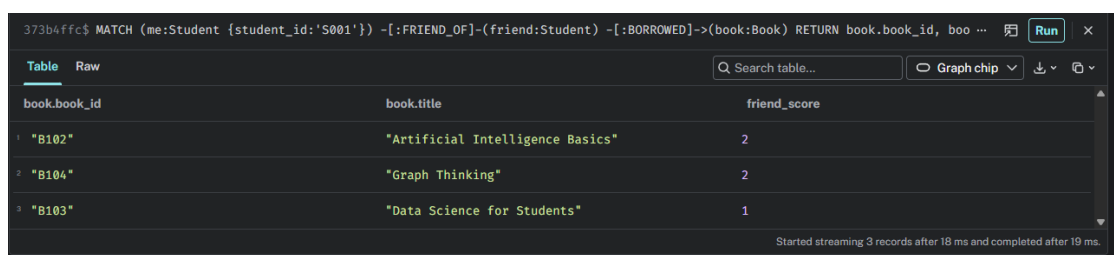
```
Student {student_id:'S001'})
```

```
-[:FRIEND_OF]-(friend:Student)
```

```
-[:BORROWED]->(book:Book)
```

```
RETURN book.book_id, book.title, count(DISTINCT friend) AS friend_score
```

```
ORDER BY friend_score DESC, book.title;
```



book.book_id	book.title	friend_score
"B102"	"Artificial Intelligence Basics"	2
"B104"	"Graph Thinking"	2
"B103"	"Data Science for Students"	1

Query แบบครบสายความคิด

```
MATCH (me:Student {student_id:'S001'})
```

```
-[:FRIEND_OF]-(friend:Student)
```

```
-[:BORROWED]->(book:Book)
```

```
WHERE NOT EXISTS {
```

```
  MATCH (me)-[:BORROWED]->(book)
```

```
}
```

```
RETURN book.book_id,
```

```
  book.title,
```

```
  count(DISTINCT friend) AS friend_score
```

```
ORDER BY friend_score DESC, book.title;
```

373b4ffc\$ MATCH (me:Student {student_id:'S001'}) -[:FRIEND_OF]-(friend:Student) -[:BORROWED]->(book:Book) WHERE NOT EXISTS { MATCH ... } Run ×

Table	Raw
book_book_id	book_title
"B102"	"Artificial Intelligence Basics"
"B104"	"Graph Thinking"
"B103"	"Data Science for Students"

Started streaming 3 records in less than 1ms and completed after 1 ms.

ดูStudent ทั้งหมด

MATCH (s:Student) RETURN s.name;

373b4ffc\$ MATCH (s:Student) RETURN s.name; Run ×

Table	Raw
s.name	
"Anan"	
"Mali"	
"Somchai"	
"Nida"	

Started streaming 4 records after 11 ms and completed after 12 ms.

ดูหนังสือที่S001 ยืม

MATCH (s:Student)-[:BORROWED]->(b:Book) WHERE s.student_id='S001' RETURN b.title;

373b4ffc\$ MATCH (s:Student)-[:BORROWED]->(b:Book) WHERE s.student_id='S001' RETURN b.title; Run ×

Table	Raw
b.title	
"Python for Beginners"	

Started streaming 1 record after 19 ms and completed after 20 ms.

ดูเพื่อนของ S001

MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(f:Student) RETURN f.name;

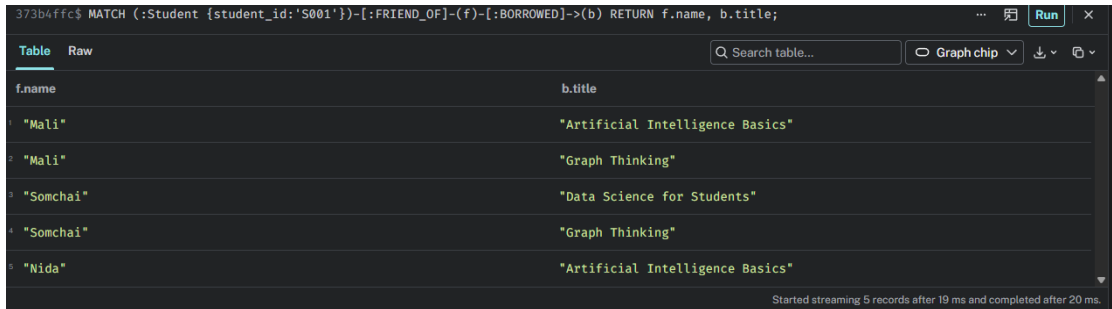
373b4ffc\$ MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(f:Student) RETURN f.name; Run ×

Table	Raw
f.name	
"Mali"	
"Somchai"	
"Nida"	

Started streaming 3 records after 18 ms and completed after 19 ms.

ดูหนังสือที่เพื่อนยืม

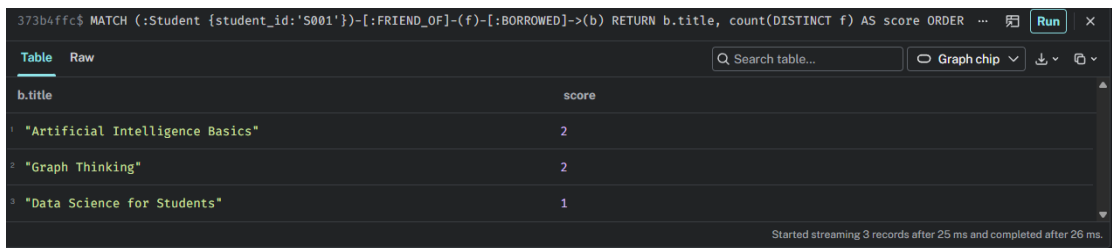
MATCH (:Student {student_id:'S001'})-[:FRIEND_OF]-(f)-[:BORROWED]->(b) RETURN f.name, b.title;



f.name	b.title
"Mali"	"Artificial Intelligence Basics"
"Mali"	"Graph Thinking"
"Somchai"	"Data Science for Students"
"Somchai"	"Graph Thinking"
"Nida"	"Artificial Intelligence Basics"

นับคะแนนหนังสือจากเพื่อน

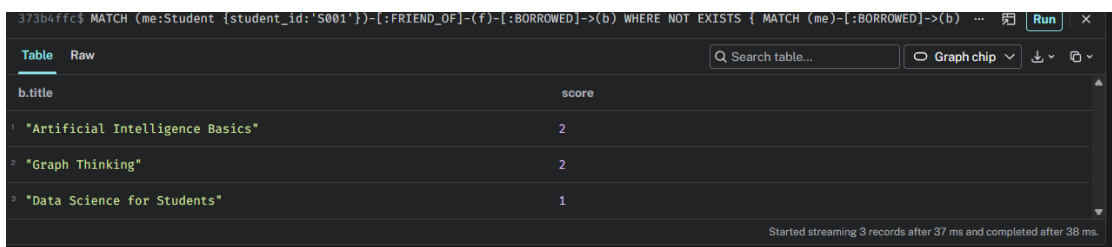
MATCH (:Student {student_id:'S001'})-[:FRIEND_OF]-(f)-[:BORROWED]->(b) RETURN b.title, count(DISTINCT f) AS score ORDER BY score DESC;



b.title	score
"Artificial Intelligence Basics"	2
"Graph Thinking"	2
"Data Science for Students"	1

แนะนำ เฉพาะหนังสือที่S001 ยังไม่เคยยืม

MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(f)-[:BORROWED]->(b) WHERE NOT EXISTS { MATCH (me)-[:BORROWED]->(b) } RETURN b.title, count(DISTINCT f) AS score ORDER BY score DESC;



b.title	score
"Artificial Intelligence Basics"	2
"Graph Thinking"	2
"Data Science for Students"	1

School Club Recommendation

กำหนด Node และ Relationship ที่จำเป็นสำหรับระบบชมรม

Node Labels:

:Student แทน นักเรียน (Property: student_id, name)

:Club แทน ชมรม (Property: club_id, name)

Relationships:

[FRIEND_OF] เชื่อมระหว่าง Student กับ Student (การเป็นเพื่อนกัน)

[MEMBER_OF] เชื่อมจาก Student ไปยัง Club (การเป็นสมาชิกชมรม)

สร้างข้อมูลตัวอย่าง (Student 5 คน และ Club 3 ชมรม)

CREATE

// สร้าง Student 5 คน

(s1:Student {student_id: 'S001', name: 'Anan'}),

(s2:Student {student_id: 'S002', name: 'Mali'}),

(s3:Student {student_id: 'S003', name: 'Somchai'}),

(s4:Student {student_id: 'S004', name: 'Nida'}),

(s5:Student {student_id: 'S005', name: 'Kanda'}),

// สร้าง Club 3 ชมรม

(c1:Club {club_id: 'C01', name: 'Coding Club'}),

(c2:Club {club_id: 'C02', name: 'Robotics Club'}),

(c3:Club {club_id: 'C03', name: 'Board Game Club'}),

// สร้างความเป็นเพื่อน (S001 เป็นเพื่อนกับ S002, S003, S004)

(s1)-[:FRIEND_OF]->(s2),

(s1)-[:FRIEND_OF]->(s3),

(s1)-[:FRIEND_OF]->(s4),

// สร้างการเป็นสมาชิกชมรม

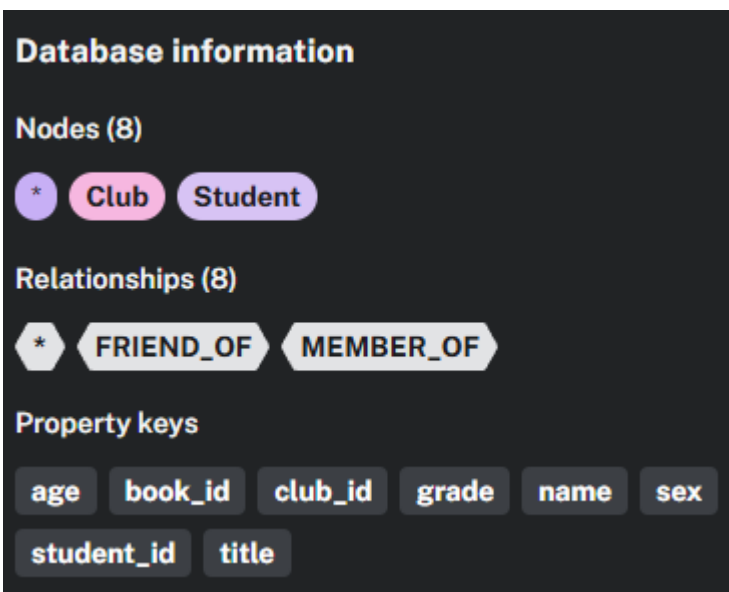
(s1)-[:MEMBER_OF]->(c3), // S001 อยู่ Board Game Club แล้ว

(s2)-[:MEMBER_OF]->(c1), // เพื่อน S002 อยู่ Coding Club

(s2)-[:MEMBER_OF]->(c2), // เพื่อน S002 อยู่ Robotics Club

(s3)-[:MEMBER_OF]->(c1), // เพื่อน S003 อยู่ Coding Club

(s4)-[:MEMBER_OF]->(c2); // เพื่อน S004 อยู่ Robotics Club



ใช้ MATCH + WHERE + RETURN เพื่อดูรวมของเพื่อน S001

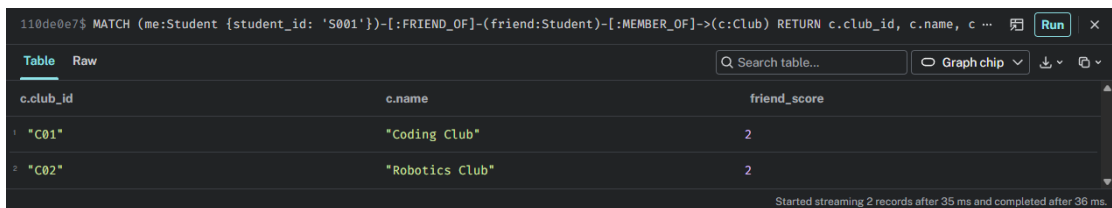
```
MATCH (me:Student)-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(c:Club) WHERE
me.student_id = 'S001' RETURN friend.name AS friend_name, c.name AS club_name;
```



friend_name	club_name
"Mali"	"Coding Club"
"Mali"	"Robotics Club"
"Somchai"	"Coding Club"
"Nida"	"Robotics Club"

ใช้ Traversal ไปยัง Club และใช้ count(DISTINCT friend) ให้คะแนน

```
MATCH (me:Student {student_id: 'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->
(c:Club) RETURN c.club_id, c.name, count(DISTINCT friend) AS friend_score ORDER BY
friend_score DESC;
```



c.club_id	c.name	friend_score
"C01"	"Coding Club"	2
"C02"	"Robotics Club"	2

ตัด Club ที่ S001 เป็น MEMBER_OF อยู่แล้วออก แล้วสร้าง Recommendation

```
MATCH (me:Student {student_id: 'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->
(c:Club) WHERE NOT EXISTS { MATCH (me)-[:MEMBER_OF]->(c) } RETURN c.club_id,
c.name AS recommended_club, count(DISTINCT friend) AS friend_score ORDER BY
friend_score DESC, c.name;
```



c.club_id	recommended_club	friend_score
"C01"	"Coding Club"	2
"C02"	"Robotics Club"	2

คำอธิบายภาษาไทย (2-3 ประโยค): “ทำไมระบบจึงแนะนำ Club อันดับ 1”

ระบบแนะนำ Coding Club (หรือชมรมที่ได้อันดับ 1) ให้กับนักเรียน SOO1 เนื่องจากมีเพื่อนของ SOO1 (ได้แก่ Mali และ Somchai) เข้าร่วมเป็นสมาชิกมากที่สุดเป็นอันดับแรก โดยระบบได้ตรวจสอบและตัดชมรมเดิมที่ SOO1 เคยเป็นสมาชิกอยู่แล้วออกไป เพื่อคัดเลือกเฉพาะชมรมใหม่ที่มีเพื่อนเข้าร่วมมากที่สุดมานำเสนอ